Guía de defensa

Prototipo no funcional · Clínicas Odontológicas · Facultad de Odontología UAM

1. Qué es lo que entrego

Un **prototipo no funcional** de la aplicación de las Clínicas Odontológicas de la Facultad de Odontología de la UAM. Está hecho en Python con Flask y plantillas HTML, y lo que muestra son **las pantallas y el flujo de trabajo**, no el sistema terminado.

Todo lo que se registra en el prototipo son **datos ficticios que duran solo la sesión**: viven en listas de Python dentro del proceso que está corriendo, así que al reiniciar el servidor la información se borra. Eso es a propósito; la base de datos definitiva será **Neon (PostgreSQL en la nube)**, y el prototipo se hizo primero para dejar claro qué pantallas y qué permisos hacen falta antes de diseñar las tablas.

Archivos del prototipo:

- `prototipo.py` — toda la lógica: rutas, roles, validaciones, bitácora y la generación del expediente en PDF.
- `templates/` — once plantillas Jinja: `base.html` (cabecera y pestañas), `entrar.html`, `menu.html`, `pacientes.html`, `razon.html`, `expediente.html`, `consentimiento.html`, `citas.html`, `salas.html`, `traspaso.html`, `bitacora.html`.
- `static/` — la hoja de estilos `estilo.css`, el logo de la UAM, el ícono y la foto de fondo.
- `pruebas.py` — las pruebas automáticas que comprueban que el flujo y los permisos funcionan.

Para ejecutarlo: `pip install flask reportlab pillow`, después `python prototipo.py` y abrir `http://localhost:5001`. Para correr las pruebas: `python pruebas.py`.

2. Las cuentas de ejemplo

Los CIF son ficticios y la contraseña de todas es `uam2026`. En el sistema real la contraseña va cifrada en la base de datos; aquí está escrita en el código a propósito, para que cualquiera pueda entrar a ver el prototipo.

CIF	Nombre	Rol
00000001	Dra. Grace Mendoza	Profesor
00000004	Dr. Gustavo Rivas	Profesor
00000002	Grace Alvarado	Estudiante
00000005	Gabriel Espinoza	Estudiante
00000007	Gina Saavedra	Facultad (admin)

3. Qué puede hacer cada rol

Estudiante (`00000002`, `00000005`): registra a *sus* pacientes, anota la razón por la que vienen, agenda sus citas y recoge la firma del paciente en el consentimiento. Solo ve a los pacientes que están a su cargo. No ve la bitácora.

Profesor (`00000001`, `00000004`): **solo revisa**. Ve a todos los pacientes de todos los estudiantes, aprueba o retira la aprobación, ve el historial de qué le hizo cada estudiante a cada paciente y revisa la bitácora. **No registra pacientes, ni citas, ni tratamientos**, y tampoco firma el consentimiento.

Facultad / admin (`00000007`): crea las cuentas, administra las salas, pasa un paciente de un estudiante a otro archivando el trabajo del anterior, y revisa la bitácora.

Esto no es solo cuestión de esconder botones. Las plantillas ocultan lo que el rol no puede hacer, pero además cada ruta de `prototipo.py` vuelve a revisar el rol antes de tocar los datos. Si alguien armara el formulario a mano, igual no pasaría: hay una prueba en `pruebas.py` ("El profesor no puede registrar ni por la fuerza") que hace exactamente ese intento y comprueba que no se registra nada.

4. El recorrido completo

1. **Entrar.** El usuario abre `http://localhost:5001` y escribe su CIF y contraseña. La función `entrar()` revisa los dos campos; si están bien guarda el CIF en la sesión y manda a `/menu`. Los botones "He olvidado mi contraseña" y "Entrar con la cuenta de la UAM" están en pantalla pero avisan "Esta opción todavía está en proceso": se muestran para que se vea que están contemplados.

2. **Inicio.** `/menu` saluda por nombre, dice con qué CIF y rol entró, lista lo que esa cuenta puede hacer y muestra la tabla de secciones que faltan.

3. **Registrar paciente.** El estudiante entra a "Mis pacientes" y llena nombre, cédula y teléfono. Al guardar, el prototipo no lo deja ahí: lo manda al paso 2.

4. **Paso 2: razón de la consulta.** La pantalla pregunta "¿por qué está el paciente?" y pide la razón, que es obligatoria. En la misma pantalla se puede agendar la **primera cita (opcional)**: fecha, hora, sala y razón de la cita. Si la razón de la cita se deja vacía, se usa la razón de la consulta.

5. **Expediente.** De ahí se cae al expediente del paciente: sus datos, la razón, el estudiante a cargo, si el docente ya aprobó, si el consentimiento ya está firmado, la tabla de trabajo por estudiante y la tabla de citas.

6. **Consentimiento firmado.** Desde el expediente se abre el consentimiento informado, con el logo de la UAM y el texto legal. **El paciente firma ahí mismo**, con el dedo en una tableta o celular, o con el mouse. Una vez firmado, la firma queda dibujada sobre la línea del paciente.

7. **Descargar el expediente en PDF.** `/pacientes/<id>/expediente.pdf` arma el expediente clínico en PDF con reportlab, con los datos, el trabajo por estudiante, las citas y la firma del paciente pegada como imagen.

5. prototipo.py por dentro

Es el archivo por el que más van a preguntar. Esto es lo importante, en orden:

Dónde	Qué hace	Qué decir si el profesor pregunta	
`CUENTAS`, `CLAVE_EJEMPLO`, `ROLES`	Las cinco cuentas de ejemplo con su nombre y su rol	Un diccionario con el CIF como llave. En el sistema real esto es la tabla de usuarios en Neon y la clave va cifrada; aquí está a la vista porque es un prototipo para mostrar.	
`MENUS`, `QUE_HACE`, `EN_PROCESO`	Las pestañas de cada rol, lo que cada rol puede hacer y las secciones pendientes	Las pestañas se arman desde el rol, no están escritas en el HTML. Por eso el estudiante no tiene pestaña de Bitácora: no es que esté escondida, es que no existe para él.	

Dónde	Qué hace	Qué decir si el profesor pregunta	
`PACIENTES`, `CITAS`, `SALAS`, `BITACORA`	Los datos de esta corrida	Cuatro listas de Python en la memoria del proceso. No es base de datos: al reiniciar quedan vacías. Es la decisión consciente que explico en la sección 7.	
`dmy()` y `desde_dmy()`	Convierten entre `2026-10-28` e `28/10/2026`	Adentro las fechas se guardan en formato ISO (`aaaa-mm-dd`), que es como las guarda PostgreSQL y como se ordenan bien comparándolas. Hacia afuera siempre se escriben y se leen dd/mm/aaaa. `dmy` está registrado también como filtro de Jinja, así que las plantillas escriben `{{ c.fecha	dmy }}`.
`quien()`	Devuelve CIF, nombre y rol de quien entró, o `None`	Un solo lugar que contesta "¿quién es este usuario?". Todas las rutas lo usan; si mañana el login cambia, se cambia aquí nada más.	
`@app.before_request exigir_sesion()`	Antes de **cada** petición, si no hay sesión manda a la pantalla de entrada	Es el portón. No tengo que acordarme de revisar la sesión en cada ruta: Flask ejecuta esto primero siempre, y las únicas excepciones son la propia pantalla de entrada y los archivos de `static`. Una ruta nueva queda protegida sin que yo haga nada.	
`paciente(pid)`	Busca un paciente, pero **solo si el usuario puede verlo**	Aquí está el control de acceso a los datos. Si quien pregunta es estudiante y el paciente no es suyo, devuelve `None`, y todas las rutas de paciente responden a `None` mandando de vuelta a la lista. Así un estudiante no puede ver el expediente de un paciente de otro escribiendo el número en la dirección.	
`mis_pacientes()`	La lista que le toca ver a cada uno	Una comprensión de lista: el estudiante ve solo los suyos, el profesor y la Facultad ven todos. Es la misma idea de "alcance" que en el sistema con base de datos será un `WHERE`.	
`anotar()` y `bitacora()`	Guardan y muestran quién hizo qué y a qué hora	`anotar()` se llama en cada acción que cambia algo: entrar, registrar, anotar la razón, agendar, aprobar, firmar, traspasar, mover salas. `bitacora()` la muestra, y lo primero que hace es devolver al menú si quien entra es estudiante.	
`pacientes()`	Registro (estudiante) y aprobación (profesor) en una sola función	Las dos acciones pasan en la misma pantalla y usan la misma tabla. Se distinguen por el campo `accion` del POST, y cada rama exige el rol correcto: `accion == "registrar" and rol == "estudiante"`, `accion == "aprobar" and rol == "profesor"`.	
`razon()`	El paso 2 del registro	Guarda la razón y, si vinieron fecha y hora, crea de una vez la primera cita. Si la fecha viene escrita pero mal, avisa y no guarda nada.	
`consentimiento()`	Recibe la firma dibujada del paciente	Acepta el POST solo si quien lo manda es el estudiante a cargo, y solo si el dato empieza con ``. Un nombre escrito a máquina no cuenta como firma.	
`citas()`	Agendar y listar citas	Solo el estudiante agenda. Valida paciente, fecha, hora y razón; rechaza fechas pasadas y rechaza una sala ya ocupada a esa misma fecha y hora.	
`salas()`	El plano de la clínica	Cada sala sale con un ✓ si está disponible o una ✗ si no. Solo la Facultad agrega salas o cambia la disponibilidad.	

Dónde	Qué hace	Qué decir si el profesor pregunta	
`traspaso()`	Cambiar un paciente de estudiante	Solo la Facultad. Cierra el período del estudiante anterior poniéndole fecha de `hasta` y el resumen obligatorio de qué se le hizo al paciente, y abre un período nuevo para el que lo recibe. Así no se pierde el historial de quién atendió qué.	
`firma_png()` y `expediente_pdf()`	Arman el expediente clínico en PDF	`firma_png()` decodifica el PNG de la firma con `base64` y lo envuelve en un `ImageReader` para que reportlab lo pueda dibujar. `expediente_pdf()` escribe el PDF en memoria con un `canvas` de reportlab y lo devuelve con la cabecera `Content-Disposition: attachment`, que es lo que le dice al navegador "esto se descarga".	

(a) La firma la dibuja el paciente, no el docente

Antes la firma era un campo de texto. Ahora, en `consentimiento.html`, hay un elemento `<canvas>` de 600×180 donde **el paciente** dibuja su firma con el dedo (en tableta o celular) o con el mouse. El dibujo se escucha con eventos `pointerdown`, `pointermove` y `pointerup`, que son los mismos tanto para dedo como para mouse, así que no hay que escribir el código dos veces.

Al enviar el formulario, `lienzo.toDataURL("image/png")` convierte el trazo en una **imagen PNG en formato dataURL** y la manda en un campo oculto; eso es lo que se guarda y lo que después se pega en el PDF. Si el paciente no dibujó nada, se manda vacío a propósito, y el servidor avisa "Falta la firma del paciente".

Dos cosas que vale aclarar antes de que las pregunten: el servidor **vuelve a revisar** que el dato empiece con ``, porque no se puede confiar en que la validación del navegador se cumplió; y **el docente no firma**. La hoja tiene una sola línea de firma, la del paciente, porque el consentimiento informado lo da el paciente, no la clínica. Hay una prueba que comprueba que en la hoja no aparece "Firma del docente".

(b) El expediente se descarga en PDF, no en .txt

Antes el expediente salía como archivo de texto. Ahora `expediente_pdf()` lo arma con **reportlab** y sale un PDF con encabezado, los datos del paciente, la razón de la consulta, el estudiante a cargo, el estado de aprobación, la tabla de trabajo por estudiante, las citas ordenadas por fecha y hora, y **la firma del paciente dibujada**, con su fecha debajo de una línea.

Detalles que se pueden defender: el PDF se escribe en memoria (`io.BytesIO`), no en un archivo en disco, porque es una descarga de una sola vez y no hay para qué dejar basura en el servidor. La función interna `linea()` lleva la cuenta de la altura y, cuando ya queda poco espacio, pasa a la hoja siguiente sola, para que el texto no se corte. Y si la firma falta o la imagen viene dañada, el PDF sale igual pero diciendo "Sin firmar" en lugar de romperse.

(c) Las fechas se escriben y se ven dd/mm/aaaa

Esto fue un cambio hecho a propósito: **no se usa el selector nativo** `<input type="date">`. El motivo es que ese control muestra la fecha según el idioma del navegador, así que en una computadora en inglés el usuario ve `mm/dd/aaaa` y en Nicaragua eso se lee mal: `03/10/2026` puede entenderse como 3 de octubre o como 10 de marzo. En una ficha clínica esa confusión no es aceptable.

La solución fue un campo de texto normal con `placeholder="dd/mm/aaaa"`, `inputmode="numeric"` para que en el celular salga el teclado de números, y un `pattern` que ayuda en el navegador. Adentro, `desde_dmy()`

convierte lo escrito a ISO (`aaaa-mm-dd`) para guardarlo, y `dmy()` lo vuelve a dd/mm/aaaa para mostrarlo. **Se guarda en ISO** porque así es como lo guarda PostgreSQL, y porque dos fechas en ISO se pueden comparar como texto para ordenarlas o para saber si una es pasada. Si la fecha viene mal escrita, `desde_dmy()` devuelve vacío y la ruta avisa "Escriba la fecha como dd/mm/aaaa" en lugar de guardar algo incorrecto.

(d) La bitácora

Cada acción que cambia algo llama a `anotar()`, que guarda la fecha y hora, el CIF, el nombre, el rol y en palabras qué se hizo ("Registró al paciente Luis Pérez (T-100)", "Aprobó a Luis Pérez", "Agendó cita de Luis Pérez el 28/10/2026 a las 09:00"). Lo más nuevo queda arriba.

La bitácora **solo la ven el profesor y la Facultad**. El estudiante no tiene esa pestaña, y si escribe `/bitacora` a mano la ruta lo devuelve al menú. Sirve para dos cosas: que el docente pueda ver qué hizo cada estudiante, y que si algo se registró mal se sepa quién y cuándo.

7. Las decisiones técnicas, una por una

¿Por qué Flask y no otro marco? Porque es Python, que es lo que se usa en clase, y porque es pequeño: cada pantalla es una función con un `@app.route` arriba y se lee de corrido. Un marco grande traería migraciones, autenticación y administración ya armadas, que para un prototipo es justamente lo que no se quiere: aquí la idea es que **se vea** cómo se decide cada permiso.

¿Por qué plantillas Jinja? Para no repetir la cabecera, las pestañas y los mensajes en cada pantalla. `base.html` tiene todo eso una sola vez y las demás plantillas la extienden con `{% extends "base.html" %}` y llenan `{% block contenido %}`. Jinja también escapa el HTML solo, así que si alguien escribe algo raro en el nombre de un paciente no se convierte en código.

¿Por qué los datos están en listas y no en base de datos? Porque el objetivo de esta etapa era **definir las pantallas, el flujo y los permisos**. Con listas se puede cambiar la forma de los datos en un minuto; con tablas ya creadas, cada cambio es una migración. Además, para la defensa es una ventaja: se reinicia el servidor y el prototipo queda limpio para la demostración siguiente. La base de datos definitiva será **Neon (PostgreSQL en la nube)**, y el prototipo ya está escrito pensando en eso: las fechas se guardan en ISO, cada paciente y cada cita ya tienen su `id` propio, y los accesos a los datos pasan todos por `paciente()` y `mis_pacientes()`, que son los dos lugares que después se van a convertir en consultas con `WHERE`.

¿Y JavaScript? Solo hay en una pantalla, la del consentimiento, y únicamente para el `<canvas>` de la firma, porque dibujar un trazo no se puede hacer con HTML solo. Todo lo demás —las validaciones, los permisos, los mensajes— lo decide Python en el servidor.

8. Las validaciones que sí existen

Conviene nombrarlas, porque es lo que separa un prototipo pensado de una maqueta vacía:

- El **nombre y la cédula** del paciente son obligatorios.
- **No se puede repetir la cédula**: si ya hay un paciente con esa cédula, avisa y no registra.
- La **razón de la consulta es obligatoria** en el paso 2.
- Las **fechas mal escritas** no se guardan: avisa que se escriban dd/mm/aaaa.
- **No se agendan citas en fechas pasadas**.
- **No se agenda una sala que ya está ocupada** a esa fecha y hora.

- Al agendar, hacen falta **paciente, fecha, hora y razón de la cita**.
- La **firma es obligatoria** para dar por firmado el consentimiento, y tiene que ser un trazo dibujado, no un nombre escrito.
- Al traspasar un paciente, el **resumen de lo que se le hizo es obligatorio**, y hay que elegir un estudiante distinto al que lo tiene.
- **No se repite el nombre de una sala**.
- Cada estudiante **solo ve y solo toca a sus pacientes**.

Todas están comprobadas en `pruebas.py`, que corre sin abrir el navegador y dice OK o FALLO por cada caso.

9. Lo que falta, y el prototipo lo dice en pantalla

La pantalla de inicio tiene una tabla de "Secciones que faltan", marcadas "En proceso". No están escondidas: se declaran.

Sección	Para qué sirve
Odontograma	Foto presencial y odontograma digital
Tratamientos y cobros	Procedimientos, tarifario y abonos
Inventario	Materiales de la clínica
Base de datos en Neon	Guardar todo en PostgreSQL en la nube en lugar de solo la sesión

También están en pantalla pero sin funcionar, y avisando que están en proceso, "He olvidado mi contraseña" y "Entrar con la cuenta de la UAM".

10. Preguntas probables y cómo responderlas

¿Por qué no tiene base de datos?

Porque esta etapa era para definir las pantallas, el flujo y los permisos, y con listas en memoria eso se puede cambiar en minutos. La base de datos definitiva va a ser Neon, que es PostgreSQL en la nube. El prototipo ya está preparado: las fechas se guardan en ISO como las guarda PostgreSQL, cada paciente y cada cita tienen su `id`, y todo el acceso a los datos pasa por dos funciones, `paciente()` y `mis_pacientes()`, que son las que se van a convertir en consultas SQL.

¿Se pierden los datos?

Sí, y es a propósito. Los pacientes, las citas y la bitácora están en listas de Python dentro del proceso que corre; al reiniciar quedan vacías. Para una demostración es una ventaja: se reinicia y queda limpio. Como son datos ficticios, no se pierde nada real.

¿Cómo se protege la información del paciente?

Con tres capas. Primera: `before_request` revisa en **cada** petición que haya sesión; sin entrar no se ve ninguna pantalla. Segunda: el rol decide qué pestañas existen y, más importante, cada ruta vuelve a revisar el rol antes de tocar los datos, así que armar el formulario a mano no sirve. Tercera: `paciente()` no entrega un paciente a un estudiante que no lo tiene a cargo, ni aunque escriba el número en la dirección. Lo que falta

para producción, y lo digo de frente: HTTPS, las contraseñas cifradas en la base de datos, y una clave de sesión de verdad en lugar de la que está escrita en el código.

¿Es legal la firma digital?

En el prototipo no pretendo que lo sea. Lo que hay es la firma manuscrita del paciente, dibujada con el dedo o el mouse y guardada como imagen, que es el equivalente de la firma en papel y es lo mismo que usan los bancos y las mensajerías para firmar al recibir. Una firma digital con valor legal pleno necesita un certificado de una entidad autorizada y un sellado de tiempo, y eso es una decisión de la Facultad, no del prototipo. Lo que sí hice es dejar el lugar exacto donde iría: el consentimiento ya guarda quién firmó y en qué fecha, y la bitácora guarda a qué hora se recogió la firma y quién la recogió.

¿Qué falta para que sea funcional?

Cuatro cosas, en orden: la base de datos en Neon, para que la información no se pierda; el odontograma, que es el corazón de la ficha clínica; los tratamientos y cobros con su tarifario; y el inventario. Además, para poder usarlo de verdad hacen falta HTTPS, las contraseñas cifradas, la entrada con la cuenta institucional de la UAM y la recuperación de contraseña.

¿Por qué el profesor no registra pacientes?

Porque en la clínica el que atiende es el estudiante y el docente supervisa. Si el docente pudiera registrar, la bitácora dejaría de servir para saber qué hizo cada estudiante. El prototipo lo sostiene en el código, no solo escondiendo botones: la rama de registro exige `rol == "estudiante"`, y hay una prueba que manda el formulario de registro como profesor y comprueba que no se registró nada.

¿Por qué no usaron el calendario del navegador para las fechas?

Porque `<input type="date">` muestra la fecha según el idioma del navegador: en una computadora en inglés sale mm/dd/aaaa. En una ficha clínica, que 03/10/2026 se pueda leer de dos maneras no es aceptable. Por eso se escribe a mano en dd/mm/aaaa, el servidor lo convierte a ISO para guardarlo, y si está mal escrita avisa en lugar de guardar una fecha equivocada.

¿Por qué una sola función atiende el registro y la aprobación?

Porque las dos acciones ocurren en la misma pantalla y usan la misma tabla. Separarlas sería repetir la carga de la lista de pacientes y el armado de las pestañas. Se distinguen por el campo `accion` del POST, y cada rama exige su rol.

¿Qué pasa si un estudiante escribe la dirección del expediente de un paciente que no es suyo?

Lo manda de vuelta a su lista de pacientes y no ve nada. La ruta pide el paciente con `paciente()`, esa función revisa que el paciente esté a cargo de quien pregunta y, si no, devuelve `None`. Hay dos pruebas que lo comprueban.